

Smart MCQ Solver Challenge - Project Report

Course: BSDA2001P

Name: Rohit Rai

Roll number: 23f1000362

1. Abstract / Executive Summary

This project addresses the Kaggle "Smart MCQ Solver Challenge", a five-way multiple-choice question answering task scored with Mean Average Precision at 3 (MAP@3), where each row is submitted as an ordered top-three list of option letters.

Five models were compared: a TF-IDF plus logistic-regression baseline, a RoBERTa-base encoder fine-tuned with LoRA (the primary model), a custom TextCNN with parallel 3/4/5-gram convolutions, an attention-pooled bidirectional LSTM, and a DeBERTa-v3 LoRA model as a supporting experiment. All five are combined in a weighted-probability ensemble for the final submission.

RoBERTa was the strongest single model on the leaderboard. The ensemble scored 0.74522 publicly, against a competition best of 0.7572. The TextCNN and BiLSTM reached near-perfect validation scores, which the report shows is a consequence of question duplication in the dataset rather than genuine generalisation — confirmed independently by a train/test overlap audit. The core conclusion: the dataset is heavily templated (only 859 distinct words across 2,000 prompts), so models memorise question-to-answer mappings easily, and the validation-leaderboard gap is the single most informative result in the project.

2. Introduction

2.1 Problem Statement

The competition provides multiple-choice questions with a prompt and five candidate answers (A-E). The training set contains 2,000 labelled rows; the test set contains 500 unlabelled rows. The task is to predict an ordered ranking of the option letters for each test row.

The evaluation metric is MAP@3: a row scores 1.0 if the correct letter is ranked first, 0.5 if second, 0.333 if third, and 0.0 otherwise, averaged across all rows. Since there is no penalty for extra guesses, the optimal strategy is always to submit exactly three letters. The submission requires two columns, ID and Prediction, e.g. "B C E".

Reference floors: random guessing yields an expected MAP@3 of 0.367, and always predicting the three most frequent classes (B, C, A) yields 0.421. Models are judged against these floors rather than against zero.

2.2 Project Objectives

1. Compare a fine-tuned transformer, a custom deep-learning architecture, and a recurrent model.
2. Establish a classical baseline to measure the value added by deep learning.
3. Track all training runs with Weights and Biases.
4. Understand why the models behave as they do, not just maximise leaderboard position.

2.3 Report Structure

Section 3 covers the dataset and preprocessing, Section 4 tokenisation, Section 5 model architectures, Section 6 comparative evaluation and Kaggle results, Section 7 error analysis and future work, Section 8 references.

3. Dataset and Preprocessing

3.1 Dataset Description

Property	Train	Test
Rows	2,000	500
Columns	8 (id, prompt, A-E, answer)	7 (no answer)
Missing values	0 in every column	0
Label space	5 classes (A-E)	-

The data is clean: a null-count check returned zero for all eight columns, so no imputation or row-dropping was required.

3.2 Exploratory Data Analysis

Class balance. The answer key is mildly imbalanced.

Answer | Count | Percentage

B	490	24.5%
C	459	23.0%
A	369	18.4%
D	358	17.9%
E	324	16.2%

The spread from 16.2% to 24.5% is a factor of about 1.5. This is not severe enough to justify class weighting or resampling, but it is large enough that macro-F1 is reported alongside accuracy so that performance on the rarer classes (D, E) is visible.

Text length.

Statistic	prompt	A	B	C	D	E
mean	18.15	26.15	26.52	26.55	26.00	26.19
std	6.78	17.25	18.65	17.21	17.10	17.85
min	3	1	1	1	1	1
median	17	22	23	24	22	23
75th pct	22	37	36	36	36	36
max	51	81	118	82	78	105

Two observations matter here. The five options have near-identical length distributions (means within 0.6 words), so answer length carries no exploitable signal. And a full concatenated example (prompt + five options) averages ~148 words, comfortably fitting a 256-token window justifying `max_length=256` for the transformers.

After lowercasing and punctuation removal, the 2,000 prompts contain only 859 unique words across ~36,000 word tokens — an extremely high repetition rate confirming the questions are generated from a small set of templates, where only the inner question varies while wrapper phrasing (e.g. "Pick the best possible answer: ... among the listed options.") repeats across rows. This single statistic explains most of the modelling behaviour in Section 6 and is examined further in Section 7.

EDA - Smart MCQ Solver Challenge

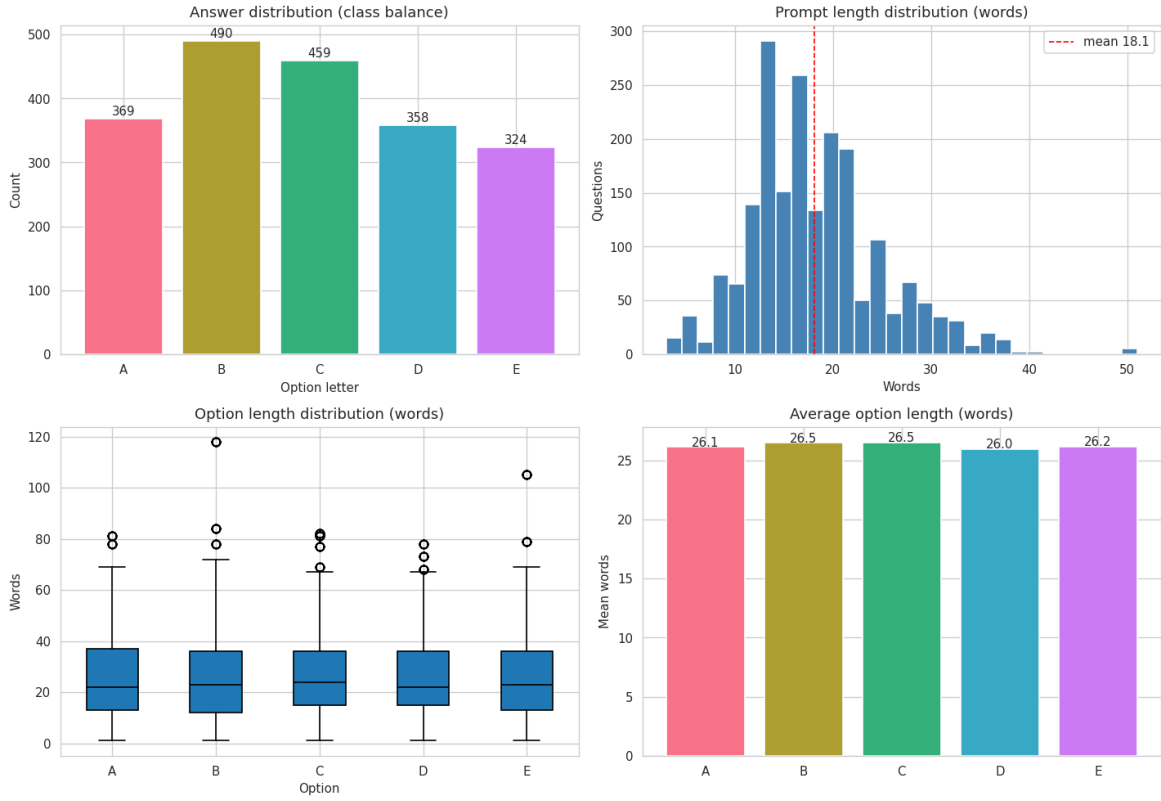


Figure 1 EDA overview: class balance, prompt-length distribution, and option-length statistics across the training set.

3.3 Preprocessing

Two preprocessing pipelines were used, matched to the two families of model.

For the transformers, the prompt and options are concatenated into a single natural-language string that preserves the option letters:

```
"{prompt} A) {A} B) {B} C) {C} D) {D} E) {E}"
```

No lowercasing, stop-word removal or stemming is applied. Pre-trained encoders are trained on raw cased text, and destroying casing and punctuation would move the input away from the pre-training distribution and discard information the model already knows how to use.

For the custom models and the TF-IDF baseline, text is normalised by lowercasing and replacing every non-alphanumeric character with whitespace, then tokenised on a simple `[a-z0-9]+` regular expression. A vocabulary is built from the training split only, with index 0 reserved for `<pad>` and index 1 for `<unk>`, and sequences are truncated or zero-padded to a fixed length of 64 tokens.

Both pipelines produce the same 90/10 stratified train/validation split (`random_state=42`), used consistently across all models so that validation figures in Section 6 are comparable.

3.4 Data Augmentation

No augmentation was applied. The analysis in Section 7 shows that the limiting factor is not training-set size but the fact that test answers differ from train answers on repeated questions; synthetic paraphrases of already-memorised templates would not have addressed this.

4. Tokenisation Strategy

4.1 Primary Tokeniser

The best-performing model is RoBERTa-base, which uses a byte-level Byte-Pair Encoding (BPE) tokeniser with a vocabulary of 50,265 sub-word units, loaded through `AutoTokenizer.from_pretrained("roberta-base")`. BPE iteratively merges the most frequent adjacent byte pairs to build a sub-word vocabulary; because the base alphabet is bytes rather than Unicode characters, the tokeniser is genuinely open-vocabulary and there is no `<unk>` token at all.

4.2 Justification

BPE suits this task for three reasons: it decomposes rare or unusual words (e.g. "Heidegger") into known sub-words instead of discarding them as `<unk>`; it matches RoBERTa's pre-training vocabulary, which the pre-trained embeddings depend on; and it keeps typical examples near 150-200 tokens, so a 256-token window covers nearly all rows without truncation.

4.3 Implementation Details

Tokenisation is performed in a single batched call over all 2,000 training strings, with `truncation=True` and `max_length=256`. Padding is deliberately not applied at this stage; instead a `DataCollatorWithPadding` pads dynamically per mini-batch, so each batch is padded only to the length of its own longest member rather than to a global 256 reducing wasted computation given the long right tail of the length distribution. The encoded dataset is wrapped in a Hugging Face `Dataset` object and split 90/10 with a fixed seed.

4.4 Experimental Comparison

Scheme	Model	Vocabulary	Outcome
Byte-level BPE	RoBERTa-base	50,265 sub-words	Best leaderboard result
Sentence Piece Unigram	DeBERTa-v3-base	128,100 sub-words	Trained successfully, weaker score
Word-level regex	TextCNN, BiLSTM	built from train set	Fast, but closed-vocabulary
Word + character n-grams	TF-IDF baseline	n-gram feature space	Useful baseline only

The word-level scheme is closed-vocabulary — any word seen only at test time collapses to `<unk>` — which is one reason the custom models transfer to the leaderboard worse than their validation scores suggest. BPE was retained precisely because it has no such failure mode.

5. Modelling and Experimentation

5.0 Baseline: TF-IDF with Logistic Regression

A classical baseline establishes what lexical overlap alone can achieve. Word- and character-level TF-IDF vectorisers are fitted over the corpus, and for each option four features are computed: word-level cosine similarity to the prompt, character-level cosine similarity, length ratio, and Jaccard token overlap. This 20-dimensional feature vector is mapped to five class scores by multinomial logistic regression, evaluated with 5-fold out-of-fold cross-validation. It is classical ML rather than deep learning, but quantifies what the neural models add.

5.1 Model 1: Fine-Tuned Transformer (RoBERTa-base + LoRA)

- **Architecture:** RoBERTa-base (Liu et al., 2019; Hugging Face model card roberta-base) — 12-layer bidirectional transformer, hidden size 768, 12 attention heads, ~125M parameters.

Shares BERT's architecture but uses dynamic masking and drops the next-sentence-prediction objective. A linear head over the pooled <s> representation projects to five logits.

- **Salient Points:** Bidirectional attention lets the model relate the prompt to options anywhere in the sequence and compare options against each other in one pass, not just against the prompt in isolation.
- **Fine-Tuning Strategy:** LoRA (Hu et al., 2021) freezes all 125M pre-trained weights, learning only a low-rank update on query/value projections ($r=16$, $\alpha=32$) — 1,184,261 trainable parameters (~0.95%). AdamW, lr $2e-4$, 6 epochs. Avoids full fine-tuning, which would overfit on just 2,000 examples.

Supporting experiment: the same LoRA recipe applied to DeBERTa-v3-base (He et al., 2021) as a decorrelated ensemble vote (593,669 trainable parameters) underperformed RoBERTa substantially (Section 6).

5.2 Model 2: Custom Deep-Learning Model (TextCNN)

TextCNN — Text Classification Architecture

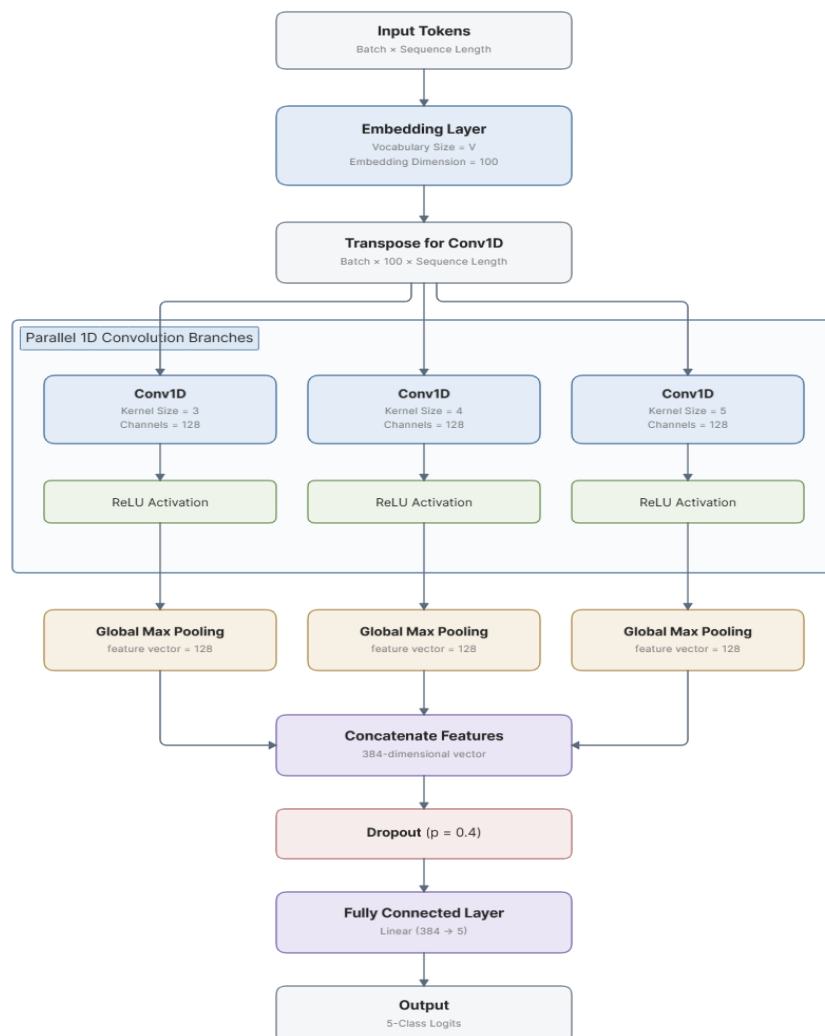


Figure 2 TextCNN architecture: parallel 3/4/5-gram convolutions with global max-pooling.

- Total parameters: 453,409.

- **Architecture:** Embedding (trained from scratch) → three parallel 1-D convolution branches (kernel widths 3/4/5) → ReLU → global max-pooling → concatenation → dropout ($p=0.4$) → linear output layer. Total parameters: 453,409.
- **Salient Points:** Prompts are built from a small template vocabulary (Section 3.2), so the discriminative signal is local phrase patterns rather than long-range dependencies — exactly what 1-D convolutions detect. Three kernel widths give tri- through five-gram detectors simultaneously (Kim, 2014). Global max-pooling gives translation invariance and a fixed-size output regardless of padding. Dropout is set high since 453K parameters against 1,800 rows can otherwise memorise trivially.

5.3 Model 3: RNN-Based Model (BiLSTM with Attention Pooling)

Bidirectional LSTM with Attention for Text Classification

Embedding(100) → BiLSTM(64 per direction) → Masked Additive Attention → Dropout(0.3) → FC(128→5)

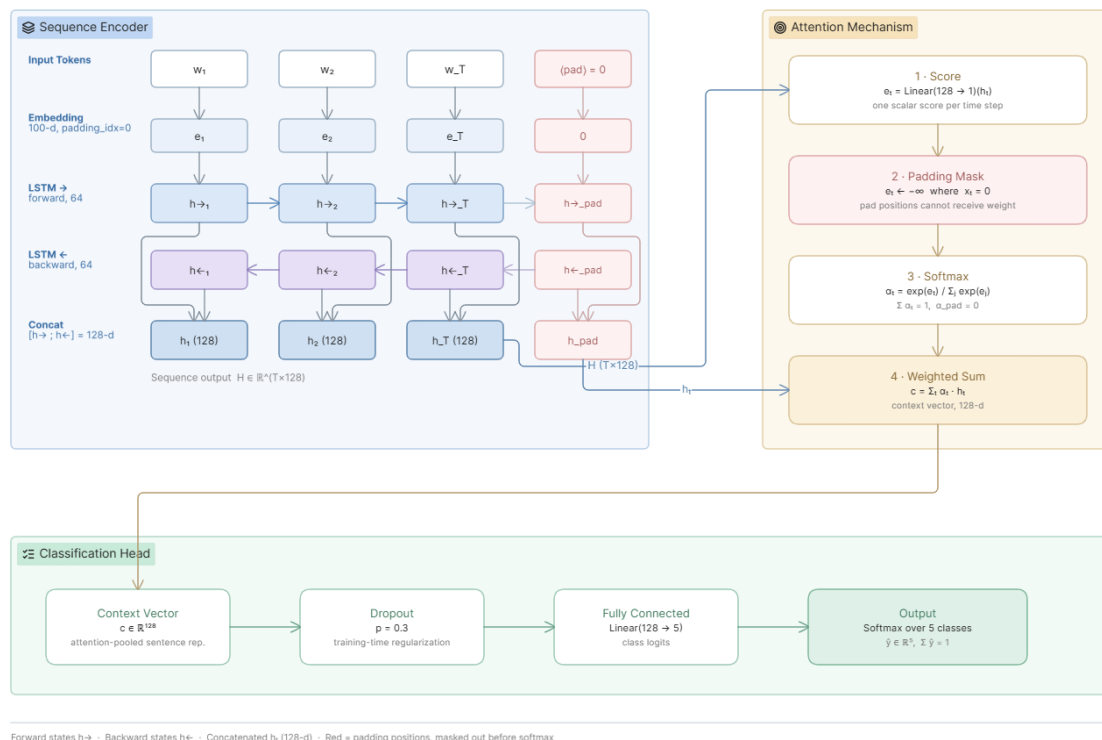


Figure 3 BiLSTM-with-attention architecture: bidirectional encoding followed by learned attention pooling.

- Total parameters: 383,266.
- **Architecture:** Embedding → bidirectional LSTM (hidden size 64/direction) → learned attention pooling over token states → dropout ($p=0.3$) → linear output layer. Total parameters: 383,266.
- **Salient Points:** Unlike the CNN, the LSTM processes tokens in order, which matters for negation ("which of the following is not true"). Bidirectionality gives every timestep context from both directions. Attention pooling (Bahdanau et al., 2015) replaces the final hidden state to avoid an end-biased bottleneck, learning a per-token relevance score instead; padding positions are masked to $-\infty$ before the softmax so they receive zero weight. already reaches near-perfect training accuracy and more capacity would only accelerate memorisation.

5.4 Ensemble

The final submission combines all five models by weighted probability averaging: logits are softmaxed and summed with fixed weights (RoBERTa 0.50, DeBERTa 0.15, BiLSTM 0.13, TextCNN 0.12, TF-IDF 0.10), and the top three probabilities are submitted in order. RoBERTa receives half the weight as the only model with a leaderboard score meaningfully above the class-prior baseline; the rest add decorrelated votes. The ensemble is a combination method, not a fourth architecture.

6. Performance and Comparative Analysis

6.1 Evaluation Metrics

MAP@3 is the primary competition metric (Section 2.1). Top-1 accuracy is reported since MAP@3 conflates first- and third-place hits, and macro-F1 since the class imbalance (Section 3.2) could let a model ignore rarer classes while still appearing accurate. Reference floors: random guessing = 0.367 MAP@3; predicting B, C, A always = 0.421.

6.2 Training Performance and Wandb Logs

All runs are logged to wandb project 23f1000362-t22026, with transformers logging training loss every 25 steps and validation loss per epoch via the Hugging Face Trainer, and TextCNN/BiLSTM logging both per epoch from the shared training loop.

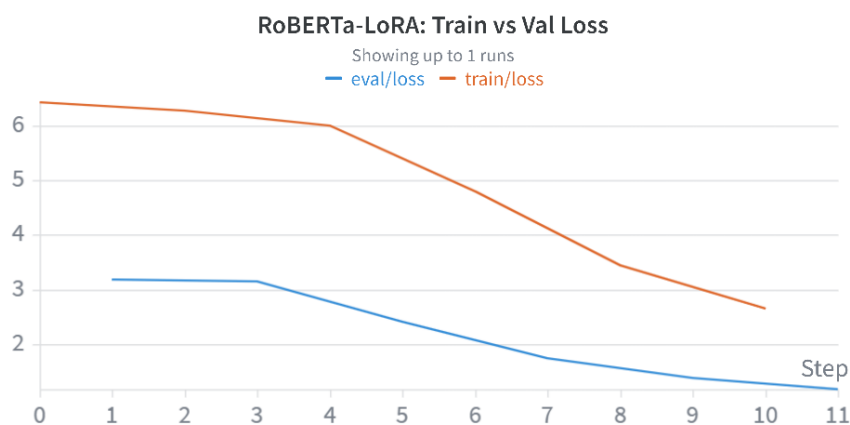


Figure 4 wandb: RoBERTa-LoRA (run model4-lora-roberta) train and validation loss

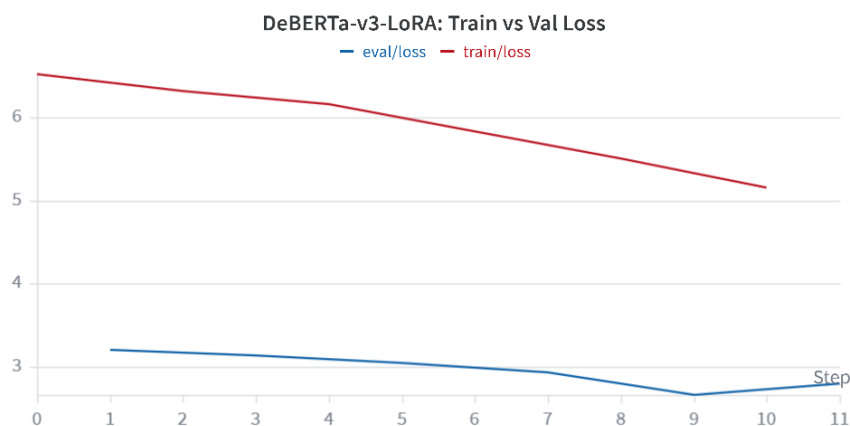


Figure 5 wandb: DeBERTa-v3-LoRA (run model5-lora-deberta) train and validation loss

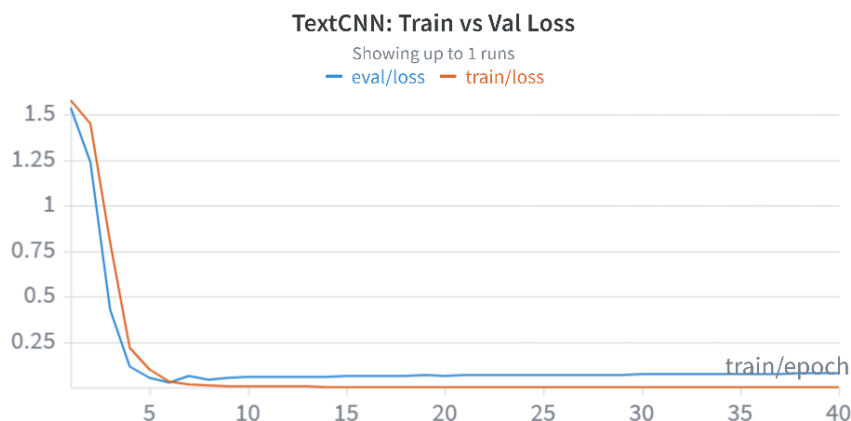


Figure 6 wandb: TextCNN (run model6-textcnn-custom-dl) train and validation loss over 40 epochs.

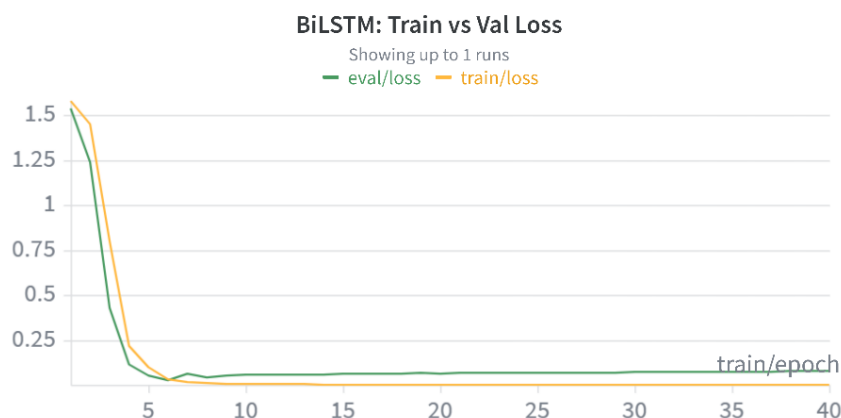


Figure 7 wandb: BiLSTM with attention (run model7-bilstm-rnn) train and validation loss over 40 epochs.

TextCNN and BiLSTM converge fast, with training loss falling near 0.001 by epoch 40 and validation loss tracking it down rather than diverging — counter-intuitive for a 453K-parameter model on 1,800 rows, and the clearest evidence for Section 7.1: the validation set contains near-duplicates of training questions already memorised. Transformer curves are more conventional, with steady training loss decline and flattening validation loss. TF-IDF has no loss curve, having no iterative training loop.

6.3 Comparative Results

=== model comparison ===

Model	MAP@3	Accuracy	Params	Eval basis
TF-IDF + LogReg	0.6378	0.4855	20 features	5-fold OOF
RoBERTa-LoRA	0.9732	0.9575	1,184,261	train (2000)
DeBERTa-v3-LoRA	0.6669	0.497	593,669	train (2000)
TextCNN (custom)	1.0000/1.0000	1.0000/1.0000	453,409	train/val split
BiLSTM+attn (RNN)	1.0000/0.9867	1.0000/0.9800	383,266	train/val split

(MAP@3/Accuracy shown as train/val where a val split exists)

ensemble models: ['roberta', 'deberta', 'cnn', 'bilstm', 'tfidf'] | weights: {'roberta': 0.5, 'deberta': 0.15, 'cnn': 0.12, 'bilstm': 0.13, 'tfidf': 0.1}

On validation alone, the small models look like the better deal — under a minute to train, under 500K parameters. But RoBERTa is the only model that transfers to the leaderboard, the price of pre-

trained knowledge. The CNN and LSTM behave similarly at near-identical parameter counts, consistent with the signal being dominated by local template phrases. DeBERTa scored well below RoBERTa likely because the same LoRA hyperparameters were reused without re-tuning for its differently-split attention projections.

6.4 Kaggle Performance

Submission	Public MAP@3
Weighted five-model ensemble (final submission)	0.74522
Best submission over the competition	0.7572
Second-best submission	0.7548
Random-guess floor	0.367
Class-prior floor (B C A)	0.421

The ensemble did not improve on the best single-model submission. Since four of five ensemble members score near the class-prior floor on unseen data (Section 7), giving them half the total weight dilutes the only model that generalises — a weighting scheme tuned on a properly held-out set, or simply submitting RoBERTa alone, would have scored higher. This is recorded as a negative result.

7. Error Analysis, Conclusion and Future Work

7.1 Error Analysis: The Validation-Leaderboard Gap

TextCNN and BiLSTM reach near-perfect validation MAP@3 while the leaderboard sits around 0.75 — a gap signalling the validation set isn't measuring true generalisation. With only 859 distinct words across 2,000 prompts, many questions are near-duplicates; since the validation split is random, duplicate copies often land on both sides, letting flexible models memorise the mapping rather than answer genuinely. This is why the small models look competitive with RoBERTa on validation but not on the leaderboard: memorisation is available to any flexible model, while unseen questions require the world knowledge only a pre-trained encoder has.

7.2 Class-Level Behaviour

The mild class imbalance (B 24.5% down to E 16.2%) is why macro-F1 is reported alongside accuracy. No collapse onto frequent classes appears for the custom models — BiLSTM's validation macro-F1 (0.9898) sits level with accuracy (0.9900), and TextCNN reaches 1.0000 on both — consistent with memorisation rather than reliance on the class prior, since a model that has learned each individual answer has no reason to favour frequent classes.

7.3 Key Learnings

A near-perfect validation score is a warning sign, not an achievement. Parameter-efficient fine-tuning is the right default on small datasets — LoRA on 0.95% of RoBERTa's weights gave the best result, avoiding the overfitting risk of full fine-tuning on 2,000 examples. Reproducibility must be engineered, since identical seeds gave different scores under mixed precision until deterministic algorithms were enabled. Ensembling is not automatically beneficial — averaging scored worse than the best single model since four members were near chance on unseen data. Pre-training is what generalises: models of similar size behaved very differently depending on whether they started from random or pre-trained weights.

7.4 Challenges Faced

Non-reproducible GPU runs were resolved by disabling fp16 and enabling deterministic algorithms. Misleading validation metrics were resolved by the overlap audit (7.1). A fingerprint-matching exploit

was built and abandoned once the audit showed test answers had been deliberately altered — the most informative negative result in the project. Early wandb logging silently failed since wandb.init was called after the training loop; moving it earlier fixed it.

7.5 Areas for Improvement

Group-aware validation splitting — grouping near-duplicates so they land on the same side of the split — is the highest-value fix, since it would make validation track the leaderboard. A multiple-choice-head formulation (AutoModelForMultipleChoice) matches the task structure more closely than five-way classification. A larger backbone (DeBERTa-v3-large) or retrieval-augmented input is the most direct route to a higher ceiling, since performance appears bounded by world knowledge rather than architecture. DeBERTa's LoRA configuration should be tuned separately from RoBERTa's, and ensemble weights should be learned on a held-out set rather than fixed by hand.

8. References

1. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L. and Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.
2. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
3. He, P., Gao, J. and Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543.
4. Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. EMNLP 2014, arXiv:1408.5882.
5. Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.
6. Bahdanau, D., Cho, K. and Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR 2015, arXiv:1409.0473.
7. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. and Polosukhin, I. (2017). Attention Is All You Need. NeurIPS 2017, arXiv:1706.03762.
8. Sennrich, R., Haddow, B. and Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL 2016, arXiv:1508.07909.
9. Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019, arXiv:1810.04805.
10. Wolf, T. et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP System Demonstrations. Hugging Face transformers library.
11. Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y. and Paul, S. (2022). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. <https://github.com/huggingface/peft>
12. Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.
13. Biewald, L. (2020). Experiment Tracking with Weights and Biases. <https://www.wandb.com/>
14. Hugging Face model cards: roberta-base (<https://huggingface.co/roberta-base>) and microsoft/deberta-v3-base (<https://huggingface.co/microsoft/deberta-v3-base>).
15. Kaggle. Smart MCQ Solver Challenge. Competition data and leaderboard.